RAG-Based Customer Support Assistant using LangGraph & HITL

HIGH LEVEL DESIGN (HLD)

Objective

The objective of this High-Level Design document is to define the overall architecture, system components, workflow interactions, and scalability considerations of the RAG-Based Customer Support Assistant. This document explains how the entire system is structured, how different components communicate with each other, and how Retrieval-Augmented Generation combined with LangGraph and HITL enables intelligent customer support automation.

1. System Overview

Problem Definition

Modern customer support systems often struggle to provide accurate, contextual, and fast responses when handling large volumes of documentation such as policy manuals, FAQs, troubleshooting guides, and product documentation. Traditional chatbots provide generic responses and fail to understand document-specific information.

The objective of this project is to design and implement a Retrieval-Augmented Generation (RAG)-based Customer Support Assistant capable of:

- Processing PDF-based knowledge documents
- Retrieving relevant information using semantic embeddings
- Generating contextual responses using Large Language Models (LLMs)
- Using LangGraph for workflow orchestration
- Applying conditional routing logic for intelligent decision-making
- Escalating complex queries to Human-in-the-Loop (HITL) support

This system acts as an intelligent AI-powered customer support assistant capable of handling real-world customer interactions.

Scope of the System

The system is designed to:

- Accept PDF knowledge documents
- Extract and preprocess text

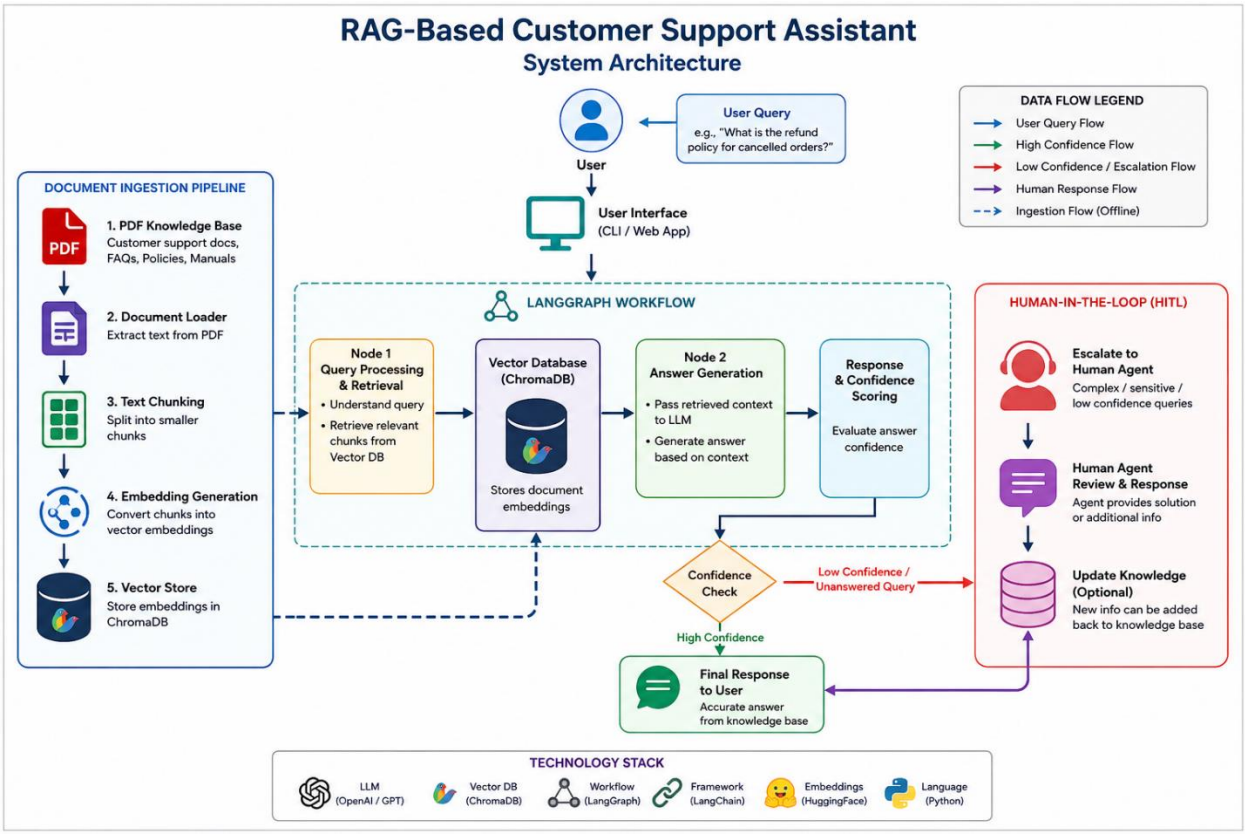
- Create vector embeddings using embedding models
- Store embeddings in ChromaDB
- Retrieve relevant document chunks based on user queries
- Generate contextual responses using an LLM
- Route responses using LangGraph workflows
- Trigger HITL escalation for uncertain or complex queries

The system does not currently support:

- Multi-language translation
- Voice-based interaction
- Multi-modal image understanding
- Long-term conversational memory

2. Architecture Diagram

System Architecture



3. Component Description

3.1 Document Loader

The document loader is responsible for reading PDF files and extracting textual content. It converts unstructured PDF data into machine-readable text format.

Responsibilities

- Load PDF documents
- Extract text page-by-page
- Handle encoding issues
- Pass extracted text to chunking module

Tools Used: PyPDFLoader from LangChain

3.2 Chunking Strategy

Chunking divides large documents into smaller manageable pieces to improve retrieval accuracy.

Chunking Parameters

- Chunk Size: 500 characters
- Chunk Overlap: 100 characters

Benefits

- Preserves semantic meaning
- Improves retrieval precision
- Reduces token overload for LLM

3.3 Embedding Model

The embedding model converts text chunks into numerical vector representations.

Model Used

- Sentence Transformers
- all-MiniLM-L6-v2

Purpose

- Semantic similarity search
- Efficient vector comparison
- Context-aware retrieval

3.4 Vector Store (ChromaDB)

ChromaDB stores embeddings and enables semantic search.

Features

- Fast vector similarity search
- Lightweight architecture
- Persistent storage support
- Open-source and developer friendly

3.5 Retriever

The retriever fetches the most relevant chunks from the vector database based on user queries.

Retrieval Flow

1. Convert query into embeddings
2. Compare with stored vectors
3. Retrieve top-k matching chunks
4. Pass chunks to LLM

3.6 LLM Processing Layer

The LLM processes retrieved chunks and generates human-like contextual responses.

Responsibilities

- Context understanding
- Answer generation
- Prompt-based reasoning
- Confidence analysis

Example Models

- OpenAI GPT
- Gemini
- Llama

3.7 Graph Workflow Engine (LangGraph)

LangGraph controls system execution using graph-based workflows.

Responsibilities

- Node execution
- State management
- Conditional routing
- Workflow orchestration

Nodes

- Input Node
- Retrieval Node
- Response Node
- HITL Node
- Output Node

3.8 Routing Layer

The routing layer decides whether the system should:

- Generate direct AI response
- Escalate to human support
- Request clarification

Routing Factors

- Confidence score
- Missing context
- Sensitive query
- Complex issue detection

3.9 Human-in-the-Loop (HITL)

The HITL module allows human intervention for low-confidence or critical queries.

Use Cases

- Billing disputes
- Legal concerns
- Unsupported requests
- Ambiguous customer queries

4. Data Flow

PDF to Answer Workflow

Step 1: PDF Ingestion

PDF documents are uploaded and processed using the document loader.

Step 2: Text Extraction

Text is extracted from the PDF.

Step 3: Chunking

The text is divided into overlapping chunks.

Step 4: Embedding Creation

Embeddings are generated for each chunk.

Step 5: Storage

Embeddings are stored in ChromaDB.

Step 6: Query Processing

User query is converted into embeddings.

Step 7: Retrieval

Relevant chunks are retrieved from ChromaDB.

Step 8: Response Generation

LLM generates contextual answer.

Step 9: Conditional Routing

System checks confidence and intent.

Step 10: HITL Escalation

If confidence is low, query is escalated.

Step 11: Final Response

Response is returned to user.

Query Lifecycle

The query lifecycle describes how a user query travels through the entire RAG system from input to final response generation.

Step-by-Step Query Lifecycle

1. The user submits a query through the interface.
2. The query is forwarded to the LangGraph workflow engine.
3. The retriever converts the query into embeddings.
4. ChromaDB performs semantic similarity search.
5. Top relevant chunks are retrieved.
6. Retrieved chunks are passed to the LLM.
7. The LLM generates a contextual response.
8. The routing layer evaluates confidence and intent.
9. If confidence is sufficient, the response is returned.
10. If confidence is low, the query is escalated to HITL.
11. Human support reviews the issue and provides assistance.
12. Final response is delivered to the user.

5. Technology Choices

Technology	**Purpose**	**Reason for Selection**
Python	Backend Development	Easy AI ecosystem integration
LangChain	RAG Pipeline	Simplifies document processing
LangGraph	Workflow Orchestration	Graph-based state management
ChromaDB	Vector Database	Lightweight and fast retrieval
Sentence Transformers	Embeddings	High semantic accuracy
OpenAI/Gemini	LLM	Natural language understanding
Streamlit	Frontend UI	Quick web app development

6. Scalability Considerations

Handling Large Documents

- Incremental ingestion
- Batch embedding creation
- Persistent vector storage

Handling High Query Volume

- Retriever caching
- Distributed vector databases
- Parallel query execution

Reducing Latency

- Smaller embedding models
- Efficient chunk retrieval
- Optimized prompt design